Agente de Inteligência Urbanística — Plano de arquitetura

Marcos Yudi Kimura · v1 · novembro de 2026

Problema

Os parâmetros de construção de Londrina estão presos em PDFs da lei de uso e ocupação do solo, difíceis de consultar.

Solução

Um pipeline que extrai os parâmetros com LLM, guarda em tabela consultável e responde perguntas citando a lei.

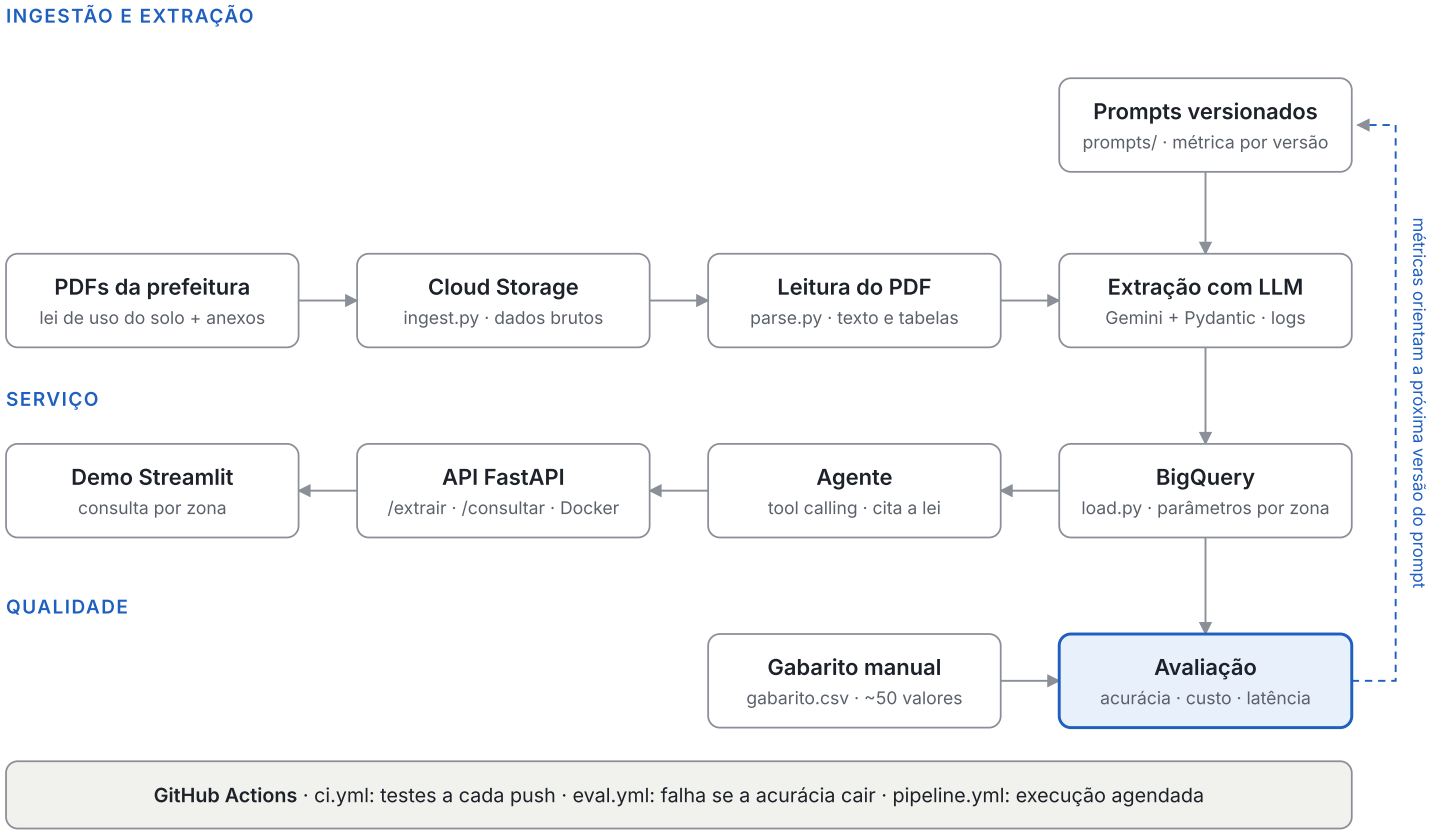
Diferencial

A extração é medida contra um gabarito manual, e o CI bloqueia qualquer mudança que piore a acurácia.

Escopo da v1

Londrina; 5 parâmetros por zona: taxa de ocupação, coeficiente de aproveitamento, altura máxima, recuo frontal e permeabilidade.

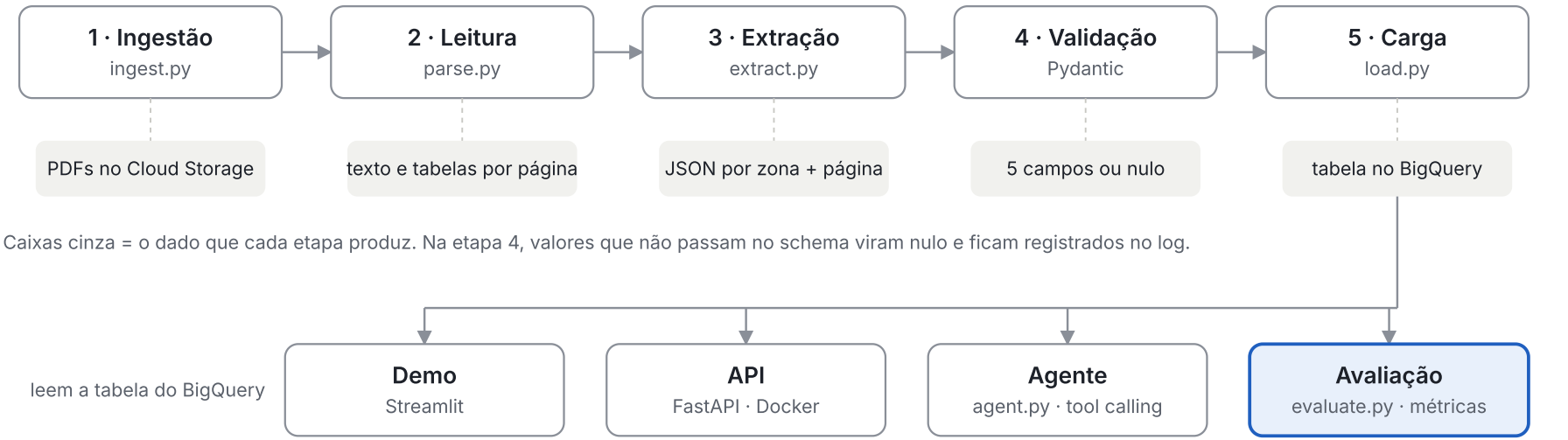
Arquitetura



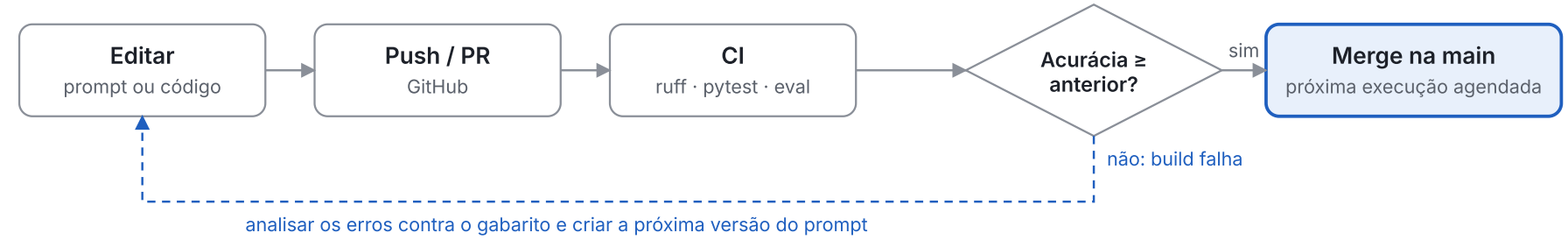
Workflow e pipeline

O pipeline transforma PDF em tabela a cada execução agendada; o workflow garante que nenhuma mudança de prompt ou código piore a extração.

PIPELINE DE DADOS · AGENDADO NO GITHUB ACTIONS



WORKFLOW DE DESENVOLVIMENTO · CADA MUDANÇA DE PROMPT OU CÓDIGO



Componentes

COMPONENTE	CÓDIGO	RESPONSABILIDADE
Prompts versionados	`prompts/`	Instruções de extração em arquivos, com a métrica de cada versão
Ingestão	`ingest.py`	Baixa a lei e os anexos e guarda os PDFs brutos no Cloud Storage
Leitura	`parse.py`	Extrai texto e tabelas por página (pdfplumber ou PyMuPDF)
Extração	`extract.py · llm.py`	Gemini com saída JSON validada pelo Pydantic; guarda a página de origem; ausente vira nulo; registra tokens, custo e latência
Armazenamento	`load.py · sql/`	Grava os 5 parâmetros por zona e as métricas de cada execução no BigQuery
Avaliação	`evaluate.py`	Compara com o gabarito: acurácia exata e por parâmetro, nulos corretos, alucinação; compara 2 a 3 modelos
Agente	`agent.py`	Tool calling: transforma a pergunta em consulta ao BigQuery e responde citando a lei
API	`api/main.py`	FastAPI com /extrair e /consultar, empacotada em Docker
Demo	`app/streamlit_app.py`	Interface pública: escolher a zona e ver os parâmetros e a página de origem
Automação	`.github/workflows/`	ci.yml (testes), eval.yml (regressão de acurácia), pipeline.yml (execução agendada)

Cronograma

FASE	DATAS	ENTREGA
MVP	9 a 15/11	Gabarito de 3 a 5 zonas, parse, extração e avaliação mínimas; README com métrica
Semana 1	16 a 22/11	Gabarito com ~50 valores, prompts versionados, avaliação no CI, comparação de modelos
Semana 2	23 a 29/11	BigQuery, métricas por execução, ingestão no Cloud Storage, pipeline agendado
Semana 3	30/11 a 6/12	Agente com tool calling, API FastAPI, Docker, testes e logs de custo
Semana 4	7 a 13/12	Demo Streamlit, README final, LinkedIn e preparação para entrevistas

Stack

Python · Gemini (`google-genai`) · Pydantic · pdfplumber / PyMuPDF · pandas · Google Cloud Storage · BigQuery · FastAPI · Docker · Streamlit · GitHub Actions · pytest · ruff

Regras do projeto

Valor ausente vira nulo, nunca um número inventado · cada valor guarda a página de origem · nenhuma chave no repositório (`.env` e secrets do GitHub) · dependências em `requirements.txt` dentro de um `.venv`